

UNIT IV – MULTIMODAL GENERATIVE AI APPLICATIONS

QUESTION 9 – AI-BASED RESUME SCREENING

Engineering Candidate Analysis and Ranking

Professional Implementation Report

Question	Develop an AI-based Resume Screening application that analyses candidate resumes and ranks them according to a given engineering job description.
Folder Name	Q9_AI_Resume_Screening
Python File	resume_screening.py
Method	TF-IDF + Cosine Similarity + Engineering Skill Matching
Interface	Streamlit Web Application

1. AIM

To develop an AI-based resume screening application that analyzes candidate resumes against an engineering job description and ranks candidates according to their relevance and skill match.

2. OBJECTIVES

- Accept an engineering job description as input.
- Accept one or more candidate resumes in PDF format.
- Extract readable text from candidate resumes.
- Represent the job description and resumes using TF-IDF.
- Calculate Cosine Similarity to measure textual relevance.
- Identify engineering skills present in the job description and resumes.
- Combine NLP relevance and skill matching into a final score.
- Rank candidates from highest to lowest suitability.

3. SOFTWARE AND HARDWARE REQUIREMENTS

- Python 3.x
- Streamlit
- Scikit-learn
- PyPDF
- Pandas
- Computer or laptop
- Internet connection not required for the core ranking algorithm

4. FOLDER AND FILE STRUCTURE

```
UNIT_IV_Multimodal_Generative_AI
├── Q9_AI_Resume_Screening
│   └── resume_screening.py
```

5. TECHNOLOGIES USED

Technology	Purpose	Role
Python	Programming	Application implementation
Streamlit	User Interface	Job description, uploads and results
PyPDF	Document Processing	Extracts text from PDF resumes
Scikit-learn	NLP	TF-IDF and Cosine Similarity
Pandas	Data Handling	Ranking table and result processing

6. SCORING METHODOLOGY

The application uses a combined score so that both general textual relevance and engineering skill coverage are considered.

Final Score = (70% × NLP Similarity) + (30% × Skill Match)

NLP Similarity is calculated using Cosine Similarity between the TF-IDF vector of the job description and the TF-IDF vector of each resume. Skill Match is calculated from the engineering skills identified in both the job description and the resume.

7. SYSTEM WORKFLOW

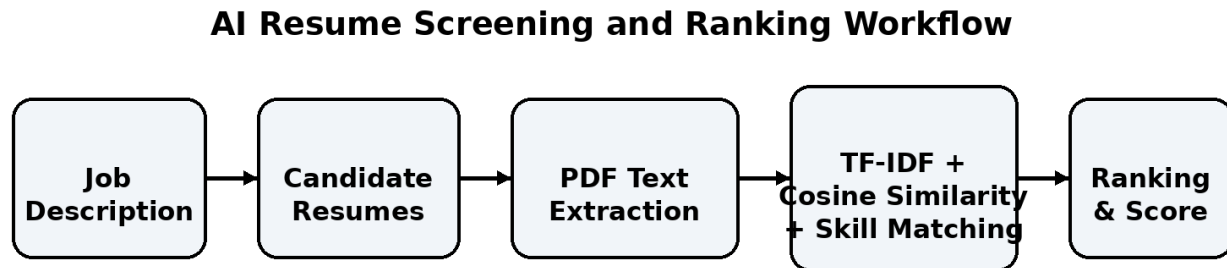


Figure 1. AI-based resume screening and ranking workflow.

8. STEP-BY-STEP IMPLEMENTATION

- Step 1: Create the Q9_AI_Resume_Screening folder.
- Step 2: Create the resume_screening.py Python file.
- Step 3: Install Streamlit, Scikit-learn, PyPDF and Pandas.
- Step 4: Create the engineering job description input box.
- Step 5: Create a multi-file PDF resume uploader.
- Step 6: Extract text from every uploaded resume.
- Step 7: Clean and normalize the extracted text.
- Step 8: Create TF-IDF vectors for the job description and each resume.
- Step 9: Calculate Cosine Similarity scores.
- Step 10: Identify engineering skills in the job description and each resume.
- Step 11: Calculate the skill-match score.
- Step 12: Combine the NLP similarity and skill-match scores.
- Step 13: Sort candidates by final score.
- Step 14: Display the ranking table and score comparison chart.
- Step 15: Allow the results to be downloaded as a CSV file.

9. COMPLETE IMPLEMENTATION CODE

```
import streamlit as st
from pypdf import PdfReader
from sklearn.feature_extraction.text import TfidfVectorizer
```

```

from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd
import re

```

```

st.set_page_config(
    page_title="AI Resume Screening",
    page_icon="📄",
    layout="wide"
)

```

```

st.title("📄 AI-Based Engineering Resume Screening")

```

```

def extract_pdf_text(pdf_file):
    reader = PdfReader(pdf_file)
    pages = []
    for page in reader.pages:
        text = page.extract_text()
        if text:
            pages.append(text)
    return "\n".join(pages)

```

```

def clean_text(text):
    text = text.lower()
    text = re.sub(r"^[^a-zA-Z0-9\s]", " ", text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()

```

```

engineering_skills = [
    "python", "java", "c", "c++", "machine learning",
    "deep learning", "artificial intelligence", "sql",
    "mysql", "data structures", "algorithms",
    "computer networks", "operating systems",
    "aws", "azure", "docker", "git", "github",
    "html", "css", "javascript", "react",
    "flask", "streamlit", "tensorflow",
    "pytorch", "scikit learn", "nlp"
]

```

```

def find_skills(text):
    text = text.lower()
    return sorted(set(
        skill for skill in engineering_skills
        if skill in text
    ))

```

```

def rank_resumes(job_description, resumes):
    documents = [clean_text(job_description)]
    resume_names = []

    for name, text in resumes:
        documents.append(clean_text(text))

```

```

        resume_names.append(name)

vectorizer = TfidfVectorizer(
    stop_words="english"
)

matrix = vectorizer.fit_transform(documents)

similarity_scores = cosine_similarity(
    matrix[0:1], matrix[1:]
)[0]

job_skills = set(
    find_skills(job_description)
)

results = []

for i, score in enumerate(similarity_scores):
    resume_skills = set(
        find_skills(resumes[i][1])
    )

    matched_skills = sorted(
        job_skills.intersection(resume_skills)
    )

    skill_score = (
        len(matched_skills) / len(job_skills)
        if job_skills else 0
    )

    final_score = (
        score * 0.70 + skill_score * 0.30
    ) * 100

    results.append({
        "Candidate": resume_names[i],
        "NLP Similarity": round(score * 100, 2),
        "Skill Match": round(skill_score * 100, 2),
        "Final Score": round(final_score, 2),
        "Matched Skills": ", ".join(matched_skills)
    })

return (
    pd.DataFrame(results)
    .sort_values("Final Score", ascending=False)
    .reset_index(drop=True)
)

st.subheader("📄 Engineering Job Description")

```

```

job_description = st.text_area(
    "Enter the job description:",
    value=(
        "We are looking for a Software Engineer with strong "
        "Python programming, SQL, machine learning, data "
        "structures, algorithms, Git, GitHub and computer "
        "networks skills."
    ),
    height=180
)

```

```

st.subheader("📄 Upload Candidate Resumes")

```

```

uploaded_resumes = st.file_uploader(
    "Upload candidate PDF resumes",
    type=["pdf"],
    accept_multiple_files=True
)

```

```

if st.button("🔍 Screen and Rank Resumes"):

```

```

    if not job_description.strip():
        st.warning("Please enter a job description.")

```

```

    elif not uploaded_resumes:
        st.warning("Please upload at least one resume.")

```

```

    else:
        resumes = []

```

```

        for resume_file in uploaded_resumes:
            text = extract_pdf_text(resume_file)
            if text.strip():
                resumes.append(
                    (resume_file.name, text)
                )

```

```

        if not resumes:
            st.error("No readable resume text was found.")

```

```

        else:
            results = rank_resumes(
                job_description, resumes
            )

```

```

            results.insert(
                0, "Rank",
                range(1, len(results) + 1)
            )

```

```

            st.success(

```

```

        "Resume screening completed successfully!"
    )

    st.subheader("🏆 Candidate Ranking")

    st.dataframe(
        results,
        width="stretch",
        hide_index=True
    )

    top_candidate = results.iloc[0]

    st.subheader("🏆 Top-Ranked Candidate")

    st.success(
        f'{top_candidate["Candidate"]} '
        f'with a final score of '
        f'{top_candidate["Final Score"]:.2f}%'
    )

    st.subheader("📊 Candidate Score Comparison")

    chart_data = results.set_index(
        "Candidate"
    )["Final Score"]

    st.bar_chart(chart_data)

    csv_data = results.to_csv(index=False)

    st.download_button(
        "📄 Download Ranking Results",
        data=csv_data,
        file_name="resume_ranking_results.csv",
        mime="text/csv"
    )

```

10. EXECUTION COMMAND

```
streamlit run ".\Q9_AI_Resume_Screening\resume_screening.py"
```

11. TEST JOB DESCRIPTION

We are looking for a Software Engineer with strong Python programming, SQL, machine learning, data structures, algorithms, Git, GitHub and computer networks skills. The candidate should have good problem-solving and software development experience.

12. IMPLEMENTATION RESULTS AND SCREENSHOTS

The following screenshots show the actual Q9 execution. The first screenshot demonstrates the job description, resume upload and successful screening message. The second screenshot shows the candidate ranking, top-ranked candidate, final score and comparison chart.



Figure 2. Q9 Streamlit interface with engineering job description, uploaded resume and successful screening.

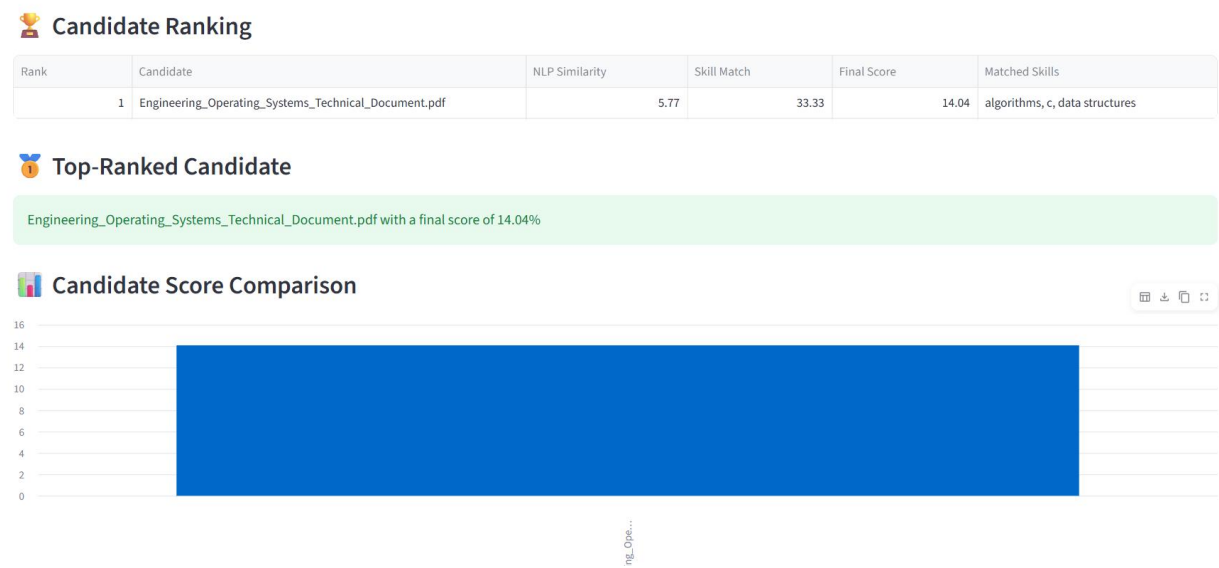


Figure 3. Candidate ranking, top-ranked candidate, final score and score comparison chart.

13. OBSERVED TEST RESULTS

Rank	Candidate	NLP Similarity	Skill Match	Final Score
1	Engineering_Operating_Systems_Technical_Document.pdf	5.77%	33.33%	14.04%

14. MATCHED SKILLS

The executed test identified the following overlapping engineering skills between the supplied job description and the uploaded document: algorithms, C, and data structures.

15. RESULT

The AI-Based Engineering Resume Screening application was successfully implemented. It accepted an engineering job description and a candidate PDF, extracted the candidate text, calculated TF-IDF similarity and engineering skill overlap, and produced a ranked screening result. In the demonstrated test, the uploaded candidate achieved an NLP similarity of 5.77%, a skill-match score of 33.33%, and a final combined score of 14.04%.

16. IMPORTANT OBSERVATION

The demonstration currently contains one uploaded PDF, so the ranking table contains one candidate. The application itself supports multiple resume uploads and will automatically rank all readable resumes when more candidates are provided. For a stronger demonstration, multiple real candidate resumes should be uploaded so that the ranking comparison contains two or more candidates.

17. ADVANTAGES

- Automatically screens multiple resumes.
- Uses objective NLP-based relevance scoring.
- Matches engineering-specific skills against job requirements.
- Produces a clear ranking table and score chart.
- Allows screening results to be downloaded as CSV.

18. LIMITATIONS

- The current scoring is based on text similarity and predefined skill matching.
- Semantic meaning beyond the available vocabulary may not be captured perfectly.
- PDFs containing scanned images may require OCR.
- A high similarity score does not guarantee real-world candidate suitability.
- Human review should still be used for final hiring decisions.

19. FUTURE ENHANCEMENTS

- Use transformer-based resume embeddings for deeper semantic matching.
- Add experience, education and certification analysis.
- Extract structured resume information automatically.
- Support DOCX and TXT resumes.
- Add configurable weights for skills, experience and education.
- Provide fairness and bias monitoring for recruitment decisions.

20. CONCLUSION

The developed AI-Based Engineering Resume Screening application demonstrates how NLP can support automated candidate screening. TF-IDF and Cosine Similarity provide a measurable relevance

score, while engineering skill matching improves alignment with the job description. The completed Streamlit application provides a practical foundation for ranking multiple engineering resumes and can be extended with more advanced semantic models and structured candidate analysis.

21. VIVA / KEY POINTS

- TF-IDF represents important words in the job description and resumes numerically.
- Cosine Similarity measures textual relevance between the job description and each resume.
- Skill matching identifies engineering skills common to the job and candidate.
- The final score combines NLP similarity and skill coverage.
- Streamlit provides the interactive resume-screening interface.